Remote Distribution — `lava-api-go` to `thinker.local`

The Lava Go API service can run on a dedicated LAN host instead of the operator's workstation. This document describes the SSH-based distribute flow used by `scripts/distribute-api-remote.sh`.

Why a remote host

- **Frees the operator's workstation** from running Postgres + the Go API + its emulator + IDE + browsers + everything else simultaneously.
- **One LAN-resident API** — Android testers running 1.2.6+ discover the API via mDNS regardless of which workstation is currently active.
- **Mirrors the Catalogizer + HelixAgent pattern** for `vasic-digital` services running on `thinker.local`.

Prerequisites (one-time setup)

1. **Passwordless SSH** from the operator's workstation to the remote host:

   ```
   # On the operator's workstation:
   ssh-copy-id milosvasic@thinker.local
   ssh milosvasic@thinker.local 'true' && echo "OK"
   ```

   (If it prints `OK` without prompting for a password, you're done.)

2. **Rootless Podman 4.x or later** on the remote host:

   ```
   ssh milosvasic@thinker.local 'podman --version'
   # Expected: podman version 4.x.y or 5.x.y
   ```

3. **`.env` keys** in the operator's workstation `.env` (gitignored — see `.env.example`):

   ```
   LAVA_API_GO_REMOTE_HOST=thinker.local
   LAVA_REMOTE_HOST_USER=milosvasic
   ```

   These default to the same values as the example, so most operators don't need to override them.

Distribute

```
# Build (if needed) + ship + boot the API on the configured remote
        host:
./scripts/distribute-api-remote.sh

# Or target a different host:
./scripts/distribute-api-remote.sh another-host.local

# Or use a different SSH user:
./scripts/distribute-api-remote.sh thinker.local --user other-user
```

The script:

1. Verifies SSH connectivity + Podman presence on the remote.
2. Resolves the lava-api-go version from `lava-api-go/internal/version/version.go`.
3. Runs `./build_and_release.sh` if the image tarball is missing.
4. Copies the OCI image tarball, `deployment/thinker/thinker.local.env`, `deployment/thinker/thinker-up.sh`, and TLS material (`server.crt` + `server.key`) to the remote.
5. Runs `thinker-up.sh` on the remote, which:
   - Creates the `lava-thinker` Podman network if missing.
   - Loads the image tarball + tags as `localhost/lava-api-go:thinker`.
   - Recreates `lava-postgres-thinker` (Postgres 16-alpine).
   - Recreates `lava-api-go-thinker` (the API binary built from `lava-api-go/`).
   - Waits for `/health` on the remote to respond.
6. Verifies `https://$REMOTE_HOST:8443/health` from the operator's workstation as a final end-to-end probe.

Verify (from the workstation)

```
curl -fsSk https://thinker.local:8443/health
# Expected: {"status":"alive"}

curl -fsSk https://thinker.local:8443/ready
# Expected: {"status":"ready"}
```

Or remote-side:

```
ssh milosvasic@thinker.local 'podman ps --filter name=lava-'
ssh milosvasic@thinker.local 'podman logs --tail 30 lava-api-go-
        thinker'
```

Tear-down

```
./scripts/distribute-api-remote.sh --tear-down thinker.local
```

Removes the `lava-api-go-thinker` + `lava-postgres-thinker` containers, the `localhost/lava-api-go:thinker` + `localhost/lava-api-go:dev` images, and the `lava-thinker` Podman network. Idempotent.

Re-deploy on every change

Per §6.P, the distribute scripts refuse to operate without a versionCode bump + matching CHANGELOG.md entry. So re-distributing after changes is:

```
# 1. Bump lava-api-go version in lava-api-go/internal/version/
        version.go
# 2. Add CHANGELOG.md entry + per-version snapshot at
#    .lava-ci-evidence/distribute-changelog/container-registry/
        <v>-<code>.md
# 3. Rebuild + redistribute:
./build_and_release.sh
./scripts/distribute-api-remote.sh
```

Constitutional bindings

- **§6.J** Anti-Bluff — `scripts/distribute-api-remote.sh` and `deployment/thinker/thinker-up.sh` use `set -euo pipefail`; failures propagate.
- **§6.B** "Container Up" is not healthy — the script verifies `/health` end-to-end, not just `podman ps`.
- **§6.K** Builds-Inside-Containers — image is produced by `build_and_release.sh` which routes through the container build path.
- **§6.M** Host-Stability — rootless Podman; no host power-management commands.
- **§6.H** Credential Security — credentials come from `.env` (gitignored); never logged.
- **§6.P** Distribution Versioning + Changelog Mandate — every distribute requires a versionCode bump + CHANGELOG entry + per-version snapshot.

Reference projects

- `../HelixAgent/scripts/deploy-full-remote-pass.sh` — fuller multi-service pattern with profile support
- `../Catalogizer/deployment/thinker.local.env` + `../Catalogizer/deployment/thinker-up.sh` — the closest analog; this Lava implementation follows the same shape